Rockchip Platform PCIe Device Virtualization Developer Guide

ID: RK-KF-YF-170

Release Version: V2.1.1

Release Date: 2026-06-05

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”)DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2026. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This solution implements peripheral virtualization based on QEMU + ARM KVM + PCIe + SMMU + VFIO. The primary objective is to enable direct access to PCIe peripherals within virtual machines. Following this guide allows mounting PCIe NVMe SSDs and establishing PCIe network interface connections inside virtual machines.

Product Version

Chip Models	Kernel Version
Rockchip chips with SMMU	4.19
RK3588	6.12

Intended Audience

This document (this guide) is primarily intended for the following engineers:

Technical Support Engineers

Software Development Engineers

Revision History

Date	Version	Author	Modification Description
2021-03-30	V1.0.0	Xue Xiaoming	Initial version
2025-09-28	V2.0.0	Xue Xiaoming	Added support for RK3588
2025-10-23	V2.1.0	Xue Xiaoming	Added support for SR-IOV
2026-06-05	V2.1.1	Xue Xiaoming	Format corrections

Table of Contents

Rockchip Platform PCIe Device Virtualization Developer Guide

1. Environment Setup
    - 1.1 Guest OS Kernel Configuration
    - 1.2 Guest OS Ramdisk Preparation
    - 1.3 HOST OS Kernel Configuration
    - 1.4 QEMU Preparation
    - 1.5 Host OS Ramdisk Preparation
2. DTS Configuration
    - 2.1 SMMU Configuration
    - 2.2 Additional PCIe Configuration
3. Execution
4. Known Issues
5. Modify the DTB

1. Environment Setup

1.1 Guest OS Kernel Configuration

Prepare the kernel

```
git clone https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux-stable.git
```

Use defconfig to generate the initial .config

```
make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- defconfig
```

Continue to enable the following configurations.

```
CONFIG_ARM64_VA_BITS_39=y
CONFIG_ARM64_VA_BITS=39
CONFIG_R8169=y
CONFIG_NVME_CORE=y
CONFIG_BLK_DEV_NVME=y
CONFIG_NVME_HWMON=y
```

Disable the following configurations.

```
CONFIG_HUGETLBFS
CONFIG_HUGETLB_PAGE
```

Use the following command to generate the guest OS kernel firmware.

```
make ARCH=arm64 CROSS_COMPILE=aarch64-linux-gnu- -j8
```

Firmware location.

```
arch/arm64/boot/Image
```

1.2 Guest OS Ramdisk Preparation

Include busybox, lspci, and ifconfig tools.

1.3 HOST OS Kernel Configuration

Generate the initial .config using the RK3588 Linux configuration:

```
make CROSS_COMPILE=aarch64-linux-gnu- ARCH=arm64 rockchip_linux_defconfig
rk3588_linux.config
```

Then enable the following configurations.

```
CONFIG_KVM=y
CONFIG_VFIO=y
CONFIG_VFIO_PCI=y
CONFIG_IOMMU_SUPPORT=y
CONFIG_ARM_SMMU_V3=y
CONFIG_VFIO_IOMMU_TYPE1=y
CONFIG_BLK_DEV_NVME=y
CONFIG_R8169=y
CONFIG_PCIE_DW=y
CONFIG_PCIE_DW_HOST=y
CONFIG_PCIE_DW_ROCKCHIP=y
CONFIG_VIRTUALIZATION=y
CONFIG_KVM_VFIO=y
CONFIG_PCI_IOV=y
```

1.4 QEMU Preparation

Compile QEMU using the Rockchip buildroot project; the resulting executable is named `qemu-system-aarch64`.

1.5 Host OS Ramdisk Preparation

Generate a ramdisk using the Rockchip buildroot project, incorporating `qemu-system-aarch64`, the Guest OS kernel Firmware, and the Guest OS ramdisk Firmware.

For Rockchip buildroot project configuration and compilation methods, refer to the buildroot usage documentation. This guide provides a pre-compiled ramdisk directory containing all required Firmware.

In an Ubuntu environment, use `abootimg-pack-initrd` to generate the `initrd.img` Firmware in the same directory as the ramdisk, and use `abootimg-unpack-initrd` to extract `initrd.img`.

To facilitate testing, the initramfs can be compiled directly into the kernel.

```
CONFIG_INITRAMFS_SOURCE="/home/xxm/initramfs"
```

2. DTS Configuration

2.1 SMMU Configuration

```
compatible = "arm,smmu-v3";
```

Mandatory configuration for matching the ARM SMMU-v3 driver.

The following Register and Interrupt configurations use the RK3588 as an example.

```
reg = <0x0 0xfc900000 0x0 0x100000>;
```

Base address and size of the ARM SMMU-v3.

```
interrupts = <GIC_SPI 369 IRQ_TYPE_EDGE_RISING>,
             <GIC_SPI 371 IRQ_TYPE_EDGE_RISING>,
             <GIC_SPI 374 IRQ_TYPE_EDGE_RISING>,
             <GIC_SPI 367 IRQ_TYPE_EDGE_RISING>;
interrupt-names = "eventq", "gerror", "priq", "cmdq-sync";
```

See Documentation/devicetree/bindings/iommu/arm,smmu-v3.txt for details.

```
#iommu-cells = <1>;
```

Must be set to 1 to describe the device's stream ID. For details, see Documentation/devicetree/bindings/iommu/arm,smmu-v3.txt.

2.2 Additional PCIe Configuration

```
&mmu600_pcie {
    status = "okay";
};
```

```
&pcie2x1l1 {
    iommu-map = <0x0 &mmu600_pcie 0x0 0x10000>;
};
&pcie3x4 {
    iommu-map = <0x0 &mmu600_pcie 0x0 0x10000>;
};
```

Create SMMU page tables for the PCIe peripheral.

3. Execution

1. Unload the host PCIe device Driver

Use lspci to obtain the device address; assume the device address is

```
   0000:00:00.0 Class 0604: Device 1d87:3588 (rev 01)
   0000:01:00.0 Class 0108: Device 1e0f:0008 (rev 01)
   0003:30:00.0 Class 0604: Device 1d87:3588 (rev 01)
   0003:31:00.0 Class 0200: Device 10ec:8168 (rev 15)
```

Execute the following command to uninstall.

```
echo 0000:01:00.0 > /sys/bus/pci/devices/0000:01:00.0/driver/unbind
echo 0003:31:00.0 > /sys/bus/pci/devices/0003:31:00.0/driver/unbind
```

2. Rebind the PCIe device to the VFIO-PCI driver

Use lspci to obtain the device vendor ID and device ID. Taking the Realtek 8169 PCIe NIC and PCIe NVMe SSD as examples, the vendor ID and device ID are 10ec:8168 and 1e0f:0008, respectively. Use the following commands to bind the devices.

```
echo 10ec 8168 > /sys/bus/pci/drivers/vfio-pci/new_id
echo 1e0f 0008 > /sys/bus/pci/drivers/vfio-pci/new_id
```

3. Run QEMU

```
qemu-system-aarch64 -nographic -monitor none -m 128 -machine virt,highmem=off -
serial stdio -cpu host -enable-kvm -net none -device vfio-pci,host=0000:01:00.0
-device vfio-pci,host=0003:31:00.0 -kernel /img/Image -initrd
/img/initramfs.cpio.gz
```

The following describes the QEMU parameters.

```
qemu-system-aarch64
```

The QEMU binary compiled by Rockchip Buildroot is placed in the bin/ directory of the host OS root filesystem.

```
-m 128
```

Allocate 128 MB of memory space for the virtual machine.

```
highmem=off
```

Use the lower 32-bit address space.

```
-device vfio-pci,host=0000:01:00.0
-device vfio-pci,host=0003:31:00.0
```

Create a VFIO device using the PCIe device address as a parameter for virtual machine access.

```
-kernel /img/Image -initrd /img/initramfs.cpio.gz
```

These are the guest OS kernel firmware and the guest OS root filesystem firmware. Place both firmware files in the img directory of the host OS root filesystem.

4. After the virtual machine has started, use the following command to configure the PCIe network interface card.

```
ifconfig eth0 up

ifconfig eth0 192.168.31.120 netmask 255.255.255.0

route add default gw 192.168.31.1
```

Configure the IP address and gateway according to the actual network environment.

5. Use the following command to test network connectivity:

```
ping 8.8.8.8
```

Under normal conditions, ping responses should be visible.

6. Configure SR-IOV

When testing SR-IOV functionality, the device address displayed by lspci is

```
00:00.0 Class 0604: Device 1d87:3588 (rev 01)
01:00.0 Class 0200: Device 8086:1521 (rev 01)
01:00.1 Class 0200: Device 8086:1521 (rev 01)
```

This is a network interface card with two ports, thus it has two PFs: `01:00.0` and `01:00.1`.

To create VFs, first obtain the maximum allowable number from sriov_totalvfs.

```
 cat /sys/bus/pci/devices/0000:01:00.0/sriov_totalvfs
 7
echo 2 >  /sys/bus/pci/devices/0000:01:00.0/sriov_numvfs
```

The following Host UART output indicates successful VF creation.

```
 pci 0000:01:10.0: [8086:1520] type 00 class 0x020000 PCIe Endpoint
pci 0000:01:10.4: [8086:1520] type 00 class 0x020000 PCIe Endpoint
```

The preceding log indicates that two VFs were created: `0000:01:10.0` and `0000:01:10.4`.

Running `lspci` again at this point will show two additional devices.

```
 00:00.0 Class 0604: Device 1d87:3588 (rev 01)
 01:00.0 Class 0200: Device 8086:1521 (rev 01)
 01:00.1 Class 0200: Device 8086:1521 (rev 01)
 01:10.0 Class 0200: Device 8086:1520 (rev 01)
01:10.4 Class 0200: Device 8086:1520 (rev 01)
```

- Load the Driver

The steps above only create the VF; the driver has not been loaded yet. Use the following command to load the driver.

```
 echo vfio-pci > /sys/bus/pci/devices/0000:01:10.0/driver_override
 echo 0000:01:10.0 > /sys/bus/pci/drivers_probe
 echo vfio-pci > /sys/bus/pci/devices/0000:01:10.4/driver_override
echo 0000:01:10.4 > /sys/bus/pci/drivers_probe
```

- Detailed Device Status

```
     lspci -v
     00:00.0 Class 0604: Device 1d87:3588 (rev 01)
             Flags: bus master, fast devsel, latency 0, IRQ 124, IOMMU group 14
             Bus: primary=00, secondary=01, subordinate=ff, sec-latency=0
             I/O behind bridge: 1000-1fff [size=4K] [16-bit]
             Memory behind bridge: f0200000-f04fffff [size=3M] [32-bit]
             Prefetchable memory behind bridge: 900000000-9000fffff [size=1M]
[32-bit]
             Expansion ROM at f0500000 [virtual] [disabled] [size=64K]
             Capabilities: [40] Power Management version 3
             Capabilities: [50] MSI: Enable+ Count=16/32 Maskable+ 64bit+
             Capabilities: [70] Express Root Port (Slot-), IntMsgNum 8
             Capabilities: [b0] MSI-X: Enable- Count=128 Masked-
             Capabilities: [100] Advanced Error Reporting
             Capabilities: [148] Secondary PCI Express
             Capabilities: [190] L1 PM Substates
             Capabilities: [1d0] Vendor Specific Information: ID=0002 Rev=4
Len=100 <?>
          Capabilities: [2d0] Vendor Specific Information: ID=0006 Rev=0 Len=018
<?>
             Kernel driver in use: pcieport

     01:00.0 Class 0200: Device 8086:1521 (rev 01)
             Subsystem: Device 8086:0002
             Flags: bus master, fast devsel, latency 0, IRQ 123, IOMMU group 15
             Memory at f0200000 (32-bit, non-prefetchable) [size=512K]
             I/O ports at 1000 [disabled] [size=32]
             Memory at f0400000 (32-bit, non-prefetchable) [size=16K]
             Expansion ROM at f0280000 [virtual] [disabled] [size=512K]
             Capabilities: [40] Power Management version 3
             Capabilities: [50] MSI: Enable- Count=1/1 Maskable+ 64bit+
             Capabilities: [70] MSI-X: Enable+ Count=10 Masked-
             Capabilities: [a0] Express Endpoint, IntMsgNum 0
             Capabilities: [100] Advanced Error Reporting
             Capabilities: [140] Device Serial Number 1c-86-0b-ff-ff-2d-d2-e8
             Capabilities: [150] Alternative Routing-ID Interpretation (ARI)
             Capabilities: [160] Single Root I/O Virtualization (SR-IOV)
             Capabilities: [1a0] Transaction Processing Hints
             Capabilities: [1c0] Latency Tolerance Reporting
          Capabilities: [1d0] Access Control Services
             Kernel driver in use: igb

     01:00.1 Class 0200: Device 8086:1521 (rev 01)
             Subsystem: Device 8086:0002
             Flags: bus master, fast devsel, latency 0, IRQ 145, IOMMU group 16
             Memory at f0300000 (32-bit, non-prefetchable) [size=512K]
             I/O ports at 1020 [disabled] [size=32]
             Memory at f0404000 (32-bit, non-prefetchable) [size=16K]
             Expansion ROM at f0380000 [virtual] [disabled] [size=512K]
             Capabilities: [40] Power Management version 3
             Capabilities: [50] MSI: Enable- Count=1/1 Maskable+ 64bit+
             Capabilities: [70] MSI-X: Enable+ Count=10 Masked-
             Capabilities: [a0] Express Endpoint, IntMsgNum 0
             Capabilities: [100] Advanced Error Reporting
             Capabilities: [140] Device Serial Number 1c-86-0b-ff-ff-2d-d2-e8
```

```
            Capabilities: [150] Alternative Routing-ID Interpretation (ARI)
            Capabilities: [160] Single Root I/O Virtualization (SR-IOV)
            Capabilities: [1a0] Transaction Processing Hints
          Capabilities: [1d0] Access Control Services
            Kernel driver in use: igb

    01:10.0 Class 0200: Device 8086:1520 (rev 01)
            Subsystem: Device 8086:0002
            Flags: fast devsel, IOMMU group 21
            Memory at 900000000 (64-bit, prefetchable) [virtual] [size=16K]
            Memory at 900020000 (64-bit, prefetchable) [virtual] [size=16K]
            Capabilities: [70] MSI-X: Enable- Count=3 Masked-
            Capabilities: [a0] Express Endpoint, IntMsgNum 0
            Capabilities: [100] Advanced Error Reporting
            Capabilities: [150] Alternative Routing-ID Interpretation (ARI)
            Capabilities: [1a0] Transaction Processing Hints
          Capabilities: [1d0] Access Control Services
            Kernel driver in use: vfio-pci

    01:10.4 Class 0200: Device 8086:1520 (rev 01)
            Subsystem: Device 8086:0002
            Flags: fast devsel, IOMMU group 22
            Memory at 900004000 (64-bit, prefetchable) [virtual] [size=16K]
            Memory at 900024000 (64-bit, prefetchable) [virtual] [size=16K]
            Capabilities: [70] MSI-X: Enable- Count=3 Masked-
            Capabilities: [a0] Express Endpoint, IntMsgNum 0
            Capabilities: [100] Advanced Error Reporting
            Capabilities: [150] Alternative Routing-ID Interpretation (ARI)
            Capabilities: [1a0] Transaction Processing Hints
            Capabilities: [1d0] Access Control Services
          Kernel driver in use: vfio-pci
```

It can be observed that devices `01:10.0` and `01:10.4` are bound to the vfio-pci Driver and have been allocated their respective BAR spaces.

7. Run QEMU to test SR-IOV

```
qemu-system-aarch64 -L /img -no-reboot -nographic -monitor none -m 256 -machine
virt,highmem=off -net none -device vfio-pci,host=0000:01:10.0 -device vfio-
pci,host=0000:01:10.4 -cpu host -enable-kvm -kernel /img/Image -initrd
/img/initramfs_guest.cpio.gz
```

The difference from Step 3 is that the device address is replaced with the VF address.

8. For a virtual machine booted via VF, lspci and ifconfig return the following information:

```
lspci -v
00:00.0 Class 0600: Device 1b36:0008
        Subsystem: Device 1af4:1100
        Flags: fast devsel

00:01.0 Class 0200: Device 8086:1520 (rev 01)
        Subsystem: Device 8086:0002
        Flags: bus master, fast devsel, latency 0
        Memory at 10000000 (64-bit, prefetchable) [size=16K]
        Memory at 10004000 (64-bit, prefetchable) [size=16K]
        Capabilities: [70] MSI-X: Enable+ Count=3 Masked-
```

```
            Capabilities: [a0] Express Root Complex Integrated Endpoint,
IntMsgNum 0
            Capabilities: [100] Advanced Error Reporting
            Capabilities: [1a0] Transaction Processing Hints
            Capabilities: [1d0] Access Control Services
            Kernel driver in use: igbvf

   00:02.0 Class 0200: Device 8086:1520 (rev 01)
            Subsystem: Device 8086:0002
            Flags: bus master, fast devsel, latency 0
            Memory at 10008000 (64-bit, prefetchable) [size=16K]
            Memory at 1000c000 (64-bit, prefetchable) [size=16K]
            Capabilities: [70] MSI-X: Enable+ Count=3 Masked-
            Capabilities: [a0] Express Root Complex Integrated Endpoint,
IntMsgNum 0
            Capabilities: [100] Advanced Error Reporting
            Capabilities: [1a0] Transaction Processing Hints
            Capabilities: [1d0] Access Control Services
            Kernel driver in use: igbvf

   ifconfig -a
   eth0      Link encap:Ethernet  HWaddr 06:7B:06:99:65:04
             BROADCAST MULTICAST  MTU:1500  Metric:1
             RX packets:0 errors:0 dropped:0 overruns:0 frame:0
             TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
             collisions:0 txqueuelen:1000
             RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

   eth1      Link encap:Ethernet  HWaddr 82:00:B8:2E:F4:64
             BROADCAST MULTICAST  MTU:1500  Metric:1
             RX packets:0 errors:0 dropped:0 overruns:0 frame:0
             TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
             collisions:0 txqueuelen:1000
             RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)

   lo        Link encap:Local Loopback
             LOOPBACK  MTU:65536  Metric:1
             RX packets:0 errors:0 dropped:0 overruns:0 frame:0
             TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
             collisions:0 txqueuelen:1000
             RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
```

There are two network interface devices emulated by VFs. The Guest kernel driver code for operating these network interfaces is igbvf. This indicates that for the VM's VF network interfaces to function correctly, the VM kernel must have the corresponding VF Driver.

Configure the network interfaces using the following commands; successful ping responses confirm that the VFs are operating normally.

```
   ifconfig eth0 up
   ifconfig eth0 192.168.31.121 netmask 255.255.255.0
   route add default gw 192.168.31.1

   ifconfig eth1 up
   ifconfig eth1 192.168.31.122 netmask 255.255.255.0
   route add default gw 192.168.31.1
```

Note that the current test environment cannot launch more than one virtual machine; therefore, only one network interface card can be tested within a single virtual machine at a time. Additionally, the network interface cards on the Host and Guest can operate simultaneously.

```
    64 bytes from 8.8.8.8: seq=141 ttl=108 time=189.385 ms
    64 bytes from 192.168.31.1: seq=103 ttl=64 time=0.655 ms
    64 bytes from 8.8.8.8: seq=142 ttl=108 time=189.099 ms
    64 bytes from 192.168.31.1: seq=104 ttl=64 time=0.597 ms
    64 bytes from 8.8.8.8: seq=143 ttl=108 time=189.428 ms
    64 bytes from 192.168.31.1: seq=105 ttl=64 time=0.584 ms
    64 bytes from 8.8.8.8: seq=144 ttl=108 time=189.002 ms
    64 bytes from 192.168.31.1: seq=106 ttl=64 time=0.547 ms
    64 bytes from 8.8.8.8: seq=145 ttl=108 time=189.032 ms
    64 bytes from 192.168.31.1: seq=107 ttl=64 time=0.532 ms
    64 bytes from 8.8.8.8: seq=146 ttl=108 time=189.673 ms
    64 bytes from 192.168.31.1: seq=108 ttl=64 time=0.630 ms
    64 bytes from 8.8.8.8: seq=147 ttl=108 time=189.086 ms
    64 bytes from 192.168.31.1: seq=109 ttl=64 time=0.611 ms
    64 bytes from 8.8.8.8: seq=148 ttl=108 time=189.729 ms
    64 bytes from 192.168.31.1: seq=110 ttl=64 time=0.650 ms
    64 bytes from 8.8.8.8: seq=149 ttl=108 time=189.450 ms
```

`8.8.8.8` and `192.168.31.1` represent the successful ping results from the Guest and Host, respectively.

9. Use the following command to test the NVMe SSD:

```
    mount /dev/nvme0n1 /tmp/
    dd if=/dev/random of=/tmp/test bs=1M count=100
```

Under normal conditions, the following information should be displayed:

```
    100+0 records in
    100+0 records out
    104857600 bytes (100.0MB) copied, 0.446296 seconds, 224.1MB/s
```

4. Known Issues

1. By default, PCIe devices in the virtual machine are configured with the coherent attribute, causing allocated DMA buffers to be marked as cacheable. Since the RK3588 does not support cache coherency, this leads to data consistency issues. To resolve this, the device must be manually configured as non-coherent during the PCIe device probe phase in the virtual machine, as shown below:

```
    dev->archdata.dma_coherent = false;
```

Taking the develop-6.12 NVMe SSD and R8169 NIC Driver as examples:

```
    git diff
    diff --git a/drivers/net/ethernet/realtek/r8169_main.c
b/drivers/net/ethernet/realtek/r8169_main.c
    index 9c601f271c02.25e443d28294 100644
    --- a/drivers/net/ethernet/realtek/r8169_main.c
```

```
   +++ b/drivers/net/ethernet/realtek/r8169_main.c
   @@ -5385,6 +5385,8 @@ static int rtl_init_one(struct pci_dev *pdev, const
struct pci_device_id *ent)
           u32 txconfig;
           u16 xid;

   +       pdev->dev.dma_coherent = false;
   +
           dev = devm_alloc_etherdev(&pdev->dev, sizeof (*tp));
           if (!dev)
                   return -ENOMEM;
   diff --git a/drivers/nvme/host/pci.c b/drivers/nvme/host/pci.c
   index 2c6d9506b172.855f725e87aa 100644
   --- a/drivers/nvme/host/pci.c
   +++ b/drivers/nvme/host/pci.c
   @@ -3417,6 +3417,8 @@ static int nvme_probe(struct pci_dev *pdev, const
struct pci_device_id *id)
           struct nvme_dev *dev;
           int result = -ENOMEM;

   +       pdev->dev.dma_coherent = false;
   +
           dev = nvme_pci_alloc_dev(pdev, id);
           if (IS_ERR(dev))
                   return PTR_ERR(dev);
```

Note that in newer kernels, such as develop-6.1 and later, the dma_coherent member has been moved to the struct device structure.

2. The file drivers/iommu/arm/arm-smmu-v3/arm-smmu-v3.c must be modified; otherwise, the following error will occur when running QEMU:

```
   vfio-pci 0003:31:00.0: probe with driver vfio-pci failed with error -22
```

This issue occurs because VFIO requires IOMMU support for cache coherence, which the SMMU on the RK3588 does not currently provide. Therefore, modifications to arm-smmu-v3.c are required:

```
   git diff drivers/iommu/arm/arm-smmu-v3/arm-smmu-v3.c
   diff --git a/drivers/iommu/arm/arm-smmu-v3/arm-smmu-v3.c
b/drivers/iommu/arm/arm-smmu-v3/arm-smmu-v3.c
   index 6bf8ecbbe0c2.9c3ea0b842e5 100644
   --- a/drivers/iommu/arm/arm-smmu-v3/arm-smmu-v3.c
   +++ b/drivers/iommu/arm/arm-smmu-v3/arm-smmu-v3.c
   @@ -2291,6 +2291,7 @@ static bool arm_smmu_capable(struct device *dev, enum
iommu_cap cap)

           switch (cap) {
           case IOMMU_CAP_CACHE_COHERENCY:
   +               return true;
                   /* Assume that a coherent TCU implies coherent TBUs */
                   return master->smmu->features & ARM_SMMU_FEAT_COHERENCY;
           case IOMMU_CAP_NOEXEC:
```

Therefore, future SMMU features must include support for cache coherence.

3. The file drivers/iommu/iommu.c must be modified; otherwise, VFs and PFs will be assigned to the same IOMMU group, which restricts all VFs/PFs to passthrough to a single virtual machine. Consequently, the following patch is required to enforce the allocation of independent IOMMU groups for each VF/PF.

```
   --- a/drivers/iommu/iommu.c
   +++ b/drivers/iommu/iommu.c
   @@ -1548,7 +1548,7 @@ struct iommu_group *pci_device_group(struct device
*dev)
                    if (!bus->self)
                            continue;

   -                if (pci_acs_path_enabled(bus->self, NULL, REQ_ACS_FLAGS))
   +                if (!pci_acs_path_enabled(bus->self, NULL, REQ_ACS_FLAGS))
                            break;

                    pdev = bus->self;
```

Future work can verify whether pairing with a Switch featuring ACS capability resolves the issue of VF/PF belonging to the same group.

4. Like other devices, the VF Driver must also set the device coherent attribute to False.

```
   --- a/drivers/net/ethernet/intel/igbvf/netdev.c
   +++ b/drivers/net/ethernet/intel/igbvf/netdev.c
   @@ -2706,6 +2706,8 @@ static int igbvf_probe(struct pci_dev *pdev, const
struct pci_device_id *ent)
            const struct igbvf_info *ei = igbvf_info_tbl[ent->driver_data];
            int err;

   +        pdev->dev.dma_coherent = false;
   +
            err = pci_enable_device_mem(pdev);
            if (err)
                    return err;
```

5. Modify the DTB

In the Guest environment, the `pdev->dev.dma_coherent` property of PCIe devices is set to true. The root cause is that the PCIe controller node in the FDT (Device Tree) generated by QEMU includes the `dma-coherent` property. Since PCIe devices inherit the `dma_coherent` attribute from the PCIe controller, this results in `pdev->dev.dma_coherent` being true. To resolve this issue, it is unnecessary to individually modify `pdev->dev.dma_coherent = false` in each PCIe Driver. Instead, simply force the PCIe controller attribute to `dma-noncoherent`. The specific method is as follows:

1. Generate the DTB

```
qemu-system-aarch64 -L /img -no-reboot -nographic -monitor none -m 256 -machine
virt,highmem=off,dumpdtb=virt64.dtb -net none  -cpu host -enable-kvm
```

The above command generates virt64.dtb on the Host.

2. Modify the dtb

```
dtc -I dtb -O dts virt64.dtb > virt64.dts
```

Modify dma-coherent to dma-noncoherent in the PCIe controller node.

3. Regenerate the dtb.

```
dtc -I dts -O dtb virt64.dts -o virt64.dtb
```

4. Specify the DTB when running QEMU

```
qemu-system-aarch64 -L /img -no-reboot -nographic -monitor none -m 256 -machine virt,highmem=off -net none -device vfio-pci,host=0000:01:10.0 -device vfio-pci,host=0000:01:10.4 -cpu host -enable-kvm -kernel /img/Image -initrd /img/initramfs_guest.cpio.gz -dtb virt64.dtb
```

With the above measures, `pdev->dev.dma_coherent` is set to false by default when enumerating PCIe devices in the Guest.

The advantage of this approach is that it eliminates the need to modify specific PCIe device Drivers in the Guest kernel.